DigiSakhi (डिजी सखी)

Complete Technical Architecture & External Viva Examination Dossier

Candidate: Sachin
Repository: github.com/sachin-saroj/digisakhi
Stack: React 19 • Node 24 • tRPC v11 • SQLite • Drizzle

1. Project Overview & Problem Statement

DigiSakhi is an empowering, bilingual (Hindi & English) cyber-safety and digital literacy platform built specifically for women participating in **Self-Help Groups (SHGs)** across India.

The Core Problem: As digital payments (UPI, AePS, micro-banking) penetrate rural and semi-urban communities, women SHG members are disproportionately targeted by cyber-fraud, phishing links, lottery scams, fake OTP requests, and personal photo extortion. Traditional cyber-security advice is overly technical, fragmented, English-only, and lacks empathetic incident disclosure mechanisms.

The DigiSakhi Solution: A lightweight, mobile-first web app providing interactive bilingual learning modules, privacy-first client-side scam inspection, stigma-free anonymous fraud reporting, 1-tap emergency helplines (1930, 1091, 112), and an administrative analytics console for SHG federations.

Pillar	Feature Capability	Engineering Implementation
Bilingual Learning	Micro-lessons on UPI Safety, Phishing Scams, Social Media Privacy, and Photos.	Bilingual JSON payloads in SQLite; instant client-side localization toggle (EN/HI).
Zero-Leakage Scam Checker	Pasted SMS/WhatsApp messages are audited for urgency, malicious URLs, and OTP traps.	**100% Client-Side Regex Heuristics.** Zero server transmission ensures complete user privacy.
Anonymous Reporting	Members submit cyber-fraud incidents without revealing their identity.	Nullable `userId` field in database. Complete de-identification at storage layer.
Quick Exit Safety	Instant 1-tap escape button redirecting browser to a neutral weather portal.	Client-side browser history replacement; protects members in hostile domestic environments.
Admin Governance	SHG Coordinators triage incident statuses and broadcast emergency notices.	Protected tRPC procedures enforcing Role-Based Access Control (RBAC).

2. Database Architecture: A to Z Deep-Dive (SQLite + Drizzle ORM)

Where is the database located and how is it executed?

The DigiSakhi database is an **embedded, file-based SQLite database** stored at the root of the project as `digisakhi.db`. It requires zero cloud hostings, zero external servers, and zero network configuration.

- **Database File:** `c:\Users\Asus\OneDrive\Documents\digisakhi\digisakhi.db`
- **Driver & Engine:** `better-sqlite3` (High-performance C++ SQLite bindings for Node.js).
- **Concurrency Engine:** Configured in **WAL (Write-Ahead Logging) Mode** via `sqlite.pragma("journal_mode = WAL")` with a `busy_timeout = 5000ms`. This permits multiple concurrent reads without blocking database writes.
- **Type-Safe ORM: Drizzle ORM** (`drizzle-orm/better-sqlite3`). Generates zero-overhead, parameterized SQL queries with full TypeScript inference.
- **Zero-Config Self-Healing:** `server/db.ts` executes automatic DDL initialization on first startup (`CREATE TABLE IF NOT EXISTS`). The app can be booted on a fresh computer and run immediately with zero manual migration commands.

End-to-End Data Pipeline Flow:

```
[User Action: Browser] ──> React Form Component (`ReportIncidentModal.tsx`)
                                 │
                                 ▼ (Type-Safe RPC Call)
[API Contract: tRPC v11] ──> `trpc.digisakhi.reportIncident.useMutation()`
                                 │
                                 ▼ (Strict Input Validation)
[Server Middleware]      ──> Zod Schema: `z.object({ description, category, anonymous })`
                                 │
                                 ▼ (ORM Query Generation)
[Drizzle ORM Engine]     ──> `db.insert(incidentReports).values(input).returning()`
                                 │ (Parameterized SQL: INSERT INTO incident_reports VALUES (?, ?, ?,
                                 ▼
[Database Engine]        ──> `better-sqlite3` writes directly to `digisakhi.db` on local SSD
                                 │
                                 ▼ (Instant Synchronous Response)
[UI Feedback]            ──> Toast Notification: "Incident Report Registered Successfully!"
```

3. Normalized Database Schema (All 8 Relational Tables)

Table Name	Module	Primary Key & Constraints	Key Columns & Description
`users`	Auth & Identity	`id` (INTEGER PK AUTOINCREMENT), `openId` (UNIQUE)	`name`, `email`, `phone`, `role` ('user'/'admin'), `shgGroup`, `preferredLanguage` ('en'/'hi').
`modules`	Education	`id` (INTEGER PK AUTOINCREMENT), `slug` (UNIQUE)	`title`, `titleHi`, `category`, `description`, `descriptionHi`, `quizData` (JSON 5 questions), `published`, `archived`.
`learning_progress`	Analytics	`id` (INTEGER PK), `UNIQUE(userId, moduleId)`	`userId`, `moduleId`, `quizScore`, `questionCount`, `completedAt`. Tracks course completion certificate status.
`quiz_attempts`	Audit Log	`id` (INTEGER PK AUTOINCREMENT)	`userId`, `moduleId`, `quizScore`, `questionCount`, `answers` (JSON array of chosen options), `completedAt`.
`incident_reports`	Fraud Reporting	`id` (INTEGER PK AUTOINCREMENT)	`userId` (Nullable for anonymity), `description`, `category`, `anonymous` (1/0), `status` ('New'/'In Review'/'Resolved').
`forum_posts`	Community	`id` (INTEGER PK AUTOINCREMENT)	`userId`, `title`, `body`, `anonymous` (1/0), `pinned` (1/0), `createdAt`. Peer scam alerts.
`forum_replies`	Community	`id` (INTEGER PK AUTOINCREMENT)	`postId`, `userId`, `body`, `anonymous` (1/0), `createdAt`. Peer advice and solutions.
`announcements`	Admin Broadcast	`id` (INTEGER PK AUTOINCREMENT)	`adminId`, `message`, `urgent` (1/0), `createdAt`. Urgent emergency security notices for SHG federation.

4. Friend's Laptop Portability Runbook (Step-by-Step Setup)

> **Why DigiSakhi runs effortlessly on any computer:** Traditional full-stack projects require installing MySQL, setting up XAMPP, creating database users, configuring port 3306, and importing `.sql` dumps. Because DigiSakhi uses an **embedded SQLite architecture**, the database travels with the folder itself!

Step-by-Step Execution on Friend's Laptop:

1. **Transfer the Project:**
   Copy the `digisakhi` folder via Pen Drive to the friend's laptop (OR clone from GitHub):

   ```
   git clone https://github.com/sachin-saroj/digisakhi.git
   cd digisakhi
   ```

   Note: If copying via Pen Drive, the `digisakhi.db` file carries over all existing users and sample complaints automatically!

2. **Install Dependencies (One-time, 60 seconds):**

   ```
   pnpm install
   ```

   (If pnpm is not installed on friend's laptop, run `npm install -g pnpm` or use `npm install`).

3. **Boot the Application:**

   ```
   pnpm dev
   ```

   The console will output: `Server running on http://localhost:3000`. On boot, `server/db.ts` automatically checks for `digisakhi.db` and initializes any missing tables in 2 milliseconds.

4. **Launch the Database Visual Studio (For the Examiner):**
   Open a second terminal window on your friend's laptop and run:

   ```
   pnpm drizzle-kit studio
   ```

   Open `https://local.drizzle.studio` in Google Chrome. All 8 database tables appear in an Excel-like interactive sheet!

5. Master Viva Questions & Answers (Database & Engineering)

Q1 "Aapke project ka database kaunsa hai aur kahan hosted hai?"

🗣️ ***Examiner Ko Yeh Boliye:***
*"Sir, hamara database lightweight, high-performance file-based **SQLite** hai. Yeh kisi 3rd party website ya external cloud par depend nahi karta, balki hamare project ke root folder mein `digisakhi.db` file ke roop mein store hota hai. Yeh 100% offline aur standalone chalta hai."*

👉 **Action on Screen:** VS Code ke File Explorer mein `digisakhi.db` file par cursor point karke dikha dein.

Q2 "Aapne SQLite kyun chuna? MySQL ya MongoDB kyun nahi?"

🗣️ ***Examiner Ko Yeh Boliye:***
"Sir, 3 solid engineering reasons hain:
*1. **Zero-Setup Portability:** XAMPP ya cloud database login karne ka koi dependency nahi. Project kisi bhi system par turant boot ho jata hai.*
*2. **Structured Relational Schema:** Hamare users, progress aur reports tightly structured relational entities hain, isliye ACID-compliant SQL best fit hai, NoSQL MongoDB ki zaroorat nahi thi.*
*3. **WAL Mode Concurrency:** Humne SQLite ko Write-Ahead Logging (WAL) mode mein configure kiya hai, jisse concurrent reads aur writes bina thread block huye instant execute hote hain."*

Q3 "Mujhe screen par database open karke aur live data save karke dikhao!" (The Golden Demo)

👉 **30-Second Golden Action Plan:**
1. Browser tab kholein: `https://local.drizzle.studio` ➔ Click on `incident_reports` table.
2. Doosre tab mein live website kholein (`http://localhost:3000`).
3. 'Report an Incident' par click karein aur submit karein: *"Fake WhatsApp electricity bill message"*.
4. Drizzle Studio tab par aakar top-right mein **Refresh (🔄)** button dabayein.
5. Naya record Row #1 par instantly display ho jayega!

🗣️ ***Examiner Ko Yeh Boliye:***
"Sir dekhiye, jaise hi frontend se submit hua, Drizzle ORM ne `server/db.ts` ke through data ko seedha `digisakhi.db` SQLite file mein write kar diya, aur Drizzle Studio ne bina server restart kiye live record display kar diya."

Q4 "Code mein actual save karne wali line kahan hai?"

👉 **Action on Screen:** Open `server/db.ts` Line 283:

```
const [created] = await db.insert(incidentReports).values(input).returning();
```

🗣️ ***Examiner Ko Yeh Boliye:***
"Sir, yeh `db.insert(incidentReports).values(input).returning()` function frontend payload ko SQL INSERT query bana kar SQLite file mein commit karta hai aur newly inserted record return karta hai."

Q5 "SQL Injection attacks se database kaise protected hai?"

🗣️ ***Examiner Ko Yeh Boliye:***
"Sir, security 2 layers par enforced hai:
*1. **Input Validation:** tRPC router boundary par Zod schema se user text ko sanitize aur validate kiya jata hai.*
*2. **Automatic Parameterized Queries:** Drizzle ORM raw SQL string concatenation kabhi nahi karta, balki automatic parameterized queries use karta hai (e.g. `VALUES (?, ?)`). Isliye SQL Injection ka risk **zero percent** hai."*

Q6 "Anonymous report mein user ki identity kaise safe rehti hai?"

🗣️ ***Examiner Ko Yeh Boliye:***
*"Sir, `drizzle/schema.ts` mein `incident_reports` table ka `userId` column **nullable** rakha gaya hai. Jab sadasya 'Submit Anonymously' chunti hai, toh backend `userId = null` insert karta hai. Is tarah database level par koi identity link hi store nahi hoti."*

6. Master Viva Questions (Core Project & Full-Stack Architecture)

Q7 **"Scam Checker kaise kaam karta hai? Kya aapne user ka personal message external AI ko bheja?"**

> 🗣️ ***Examiner Ko Yeh Boliye:***
> *"Sir, **bilkul nahi! Yeh hamari core security feature hai.***
> *Scam Checker 100% **Client-Side Local Heuristics & Regex Engine** par chalta hai. Sadasya dwara paste kiya gaya SMS ya WhatsApp message browser ke memory se bahar kabhi jata hi nahi hai. Isliye kisi third-party AI service ya server par personal bank messages leak hone ka zero chance hai."*

Q8 **"tRPC kya hota hai aur REST API ke mukable kyun use kiya?"**

> 🗣️ ***Examiner Ko Yeh Boliye:***
> *"Sir, tRPC ka matlab hai **TypeScript Remote Procedure Call**.*
> *Traditional REST API mein hume frontend par API endpoints aur JSON data types manually sync rakhne padte the. tRPC mein backend router ka TypeScript type seedha frontend React components mein infer ho jata hai. Agar backend mein koi field badli, toh frontend mein compile-time par hi error aa jata hai — zero API mismatch bugs!"*

Q9 **"Quick Exit feature ka kya role hai?"**

> 🗣️ ***Examiner Ko Yeh Boliye:***
> *"Sir, DigiSakhi rural women ke safety ke liye design ki gayi hai. Agar koi aurat harassment ya fraud report kar rahi ho aur achanak koi room mein aa jaye, toh Quick Exit button dabate hi browser history ko overwrite karke screen turant ek neutral Weather Portal par redirect ho jati hai, taaki user ki physical safety compromised na ho."*

Q10 **"Bilingual (Hindi & English) support kaise implement kiya gaya hai?"**

> 🗣️ ***Examiner Ko Yeh Boliye:***
> *"Sir, database level par har module table mein `title` & `titleHi`, aur `description` & `descriptionHi` dono columns rakhe gaye hain. Frontend par single global state hook se user bina page reload kiye instant English aur Hindi toggle kar sakta hai."*